

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage:40%

Dr

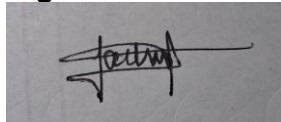
QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

I J a s h w a n t h . G . R (1 9 2 5 2 4 3 8 4) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application ; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1. Query Execution Plan and Heuristic Optimization

Consider this nested query, which returns the names of employees in the 'Research' department who earn above their department's average salary:

```
mysql> SELECT E.Ename
-> FROM Employee E
-> WHERE E.Dno = (SELECT D.Dnumber FROM Department D
->                WHERE D.Dname = 'Research')
-> AND E.Salary > (SELECT AVG(Salary) FROM Employee
->                WHERE Dno = E.Dno);
```

Step 1 - Build the initial (canonical) query tree

The parser turns the query into a canonical relational-algebra tree in a strict bottom-up fashion: base relations sit at the leaves, a Cartesian product combines them, the entire WHERE clause becomes one large selection sitting above that product, and the SELECT list becomes the projection at the root. No rewriting has happened yet:

```
1      π Ename
2      |
3      σ (Dno = Dnumber AND Dname='Research'
4         AND Salary > AVG_Salary_of_Dno)
5      |
6      × (Employee × Department × Employee_Copy)
```

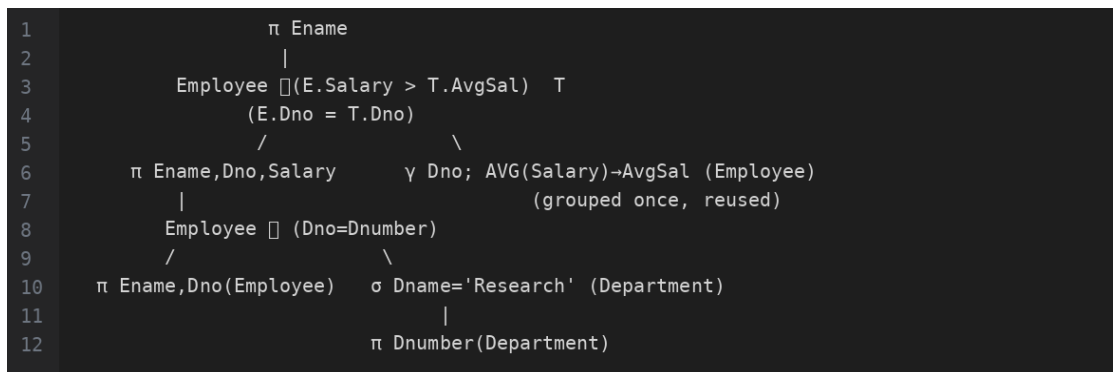
This tree is logically correct but very wasteful: it would build the full Cartesian product of Employee and Department before throwing most of it away in the selection step.

Step 2 - Apply the heuristic rewrite rules

1. Split the conjunctive selection into a chain of single-condition selections. $\sigma(a \text{ AND } b \text{ AND } c)$ is rewritten as $\sigma_a(\sigma_b(\sigma_c(...)))$, letting each condition move independently.
2. Push each selection as close to its relevant leaf as the tree allows. $\sigma(\text{Dname}='Research')$ slides down beside Department, while the join predicate $(\text{Dno} = \text{Dnumber})$ stays positioned just above the two relations it links.
3. Turn any Cartesian product immediately followed by a selection on the join predicate into an explicit join: $\sigma_{\text{join-condition}}(A \times B)$ becomes $A \text{ join_condition-join } B$. Explicit joins are cheaper since the engine can then exploit an index or a hash/merge strategy instead of enumerating every pair.
4. Reorder the joins so that whichever relation or subtree yields the smallest intermediate result runs first. Because $\sigma(\text{Dname}='Research')$ applied to Department normally returns a single row, that join is scheduled ahead of anything touching the full Employee table.

5. Move projections down as early as possible. Once only Ename, Dno, and Salary are needed further up the tree, a projection is placed right above each leaf scan so rows are narrowed before entering a join.
6. Spot repeated sub-expressions. The inner $\text{AVG}(\text{Salary}) \dots \text{GROUP BY Dno}$ computation is evaluated once per Dno group (stored as a temporary relation) instead of being recomputed for every outer row.

Step 3 - The optimized query tree



Reading the optimized tree from the bottom up: Department is narrowed to a single row first (the cheapest step, on the smallest input), the per-department average-salary subquery is computed once as a small side relation, and only afterward are the two joins carried out on already-shrunk inputs. The final projection strips every column but Ename. This is the standard System-R-style heuristic pipeline that every relational optimizer runs before cost-based join ordering even begins.

Q2. Cost-Based Optimization - Selecting a Join Algorithm

Relations Employee (E) and Department (D) are to be joined on $\text{Dno} = \text{Dnumber}$, with these statistics:

Relation	Tuples	Blocking Factor (tuples/block)	Blocks (pages)
Employee (E)	10,000	20	500 (bE)
Department (D)	125	25	5 (bD)

Buffer available: $nB = 7$ blocks. A secondary index on Dnumber (the Department table, the smaller relation) exists, with 2 index levels ($x\text{Dnumber} = 2$).

(a) Nested-Loop Join

To minimize how often the inner relation is rescanned, Department (the smaller relation) is used as the outer loop.

Using block-oriented (nested-block) nested-loop join with the 7 available buffers ($nB - 2 = 5$ blocks of the outer relation retained per pass):

```

1 Cost = bD + (bD × bE)      [naive nested loop]
2   = 5 + (5 × 500) = 5 + 2500 = 2,505 block accesses
3
4 Cost = bD + ([bD/(nB-2)] × bE)    [block nested-loop, nB=7]
5   = 5 + ([5/5] × 500) = 5 + 500 = 505 block accesses

```

(b) Sort-Merge Join

Both relations first need sorting on the join attribute (unless already sorted or indexed), after which a single linear merge pass combines them.

```

1 Sort cost (external merge sort, nB=7):
2   Employee: 2×bE×(1+[log(nB-1) [bE/nB]]) ≈ 2×500×3 = 3,000
3   Department: 2×bD×(1+[log(nB-1) [bD/nB]]) ≈ 2×5×1 = 10
4 Merge cost: bE + bD = 500 + 5 = 505
5 Total (both relations unsorted) = 3,515 block accesses
6
7 If Employee is already sorted / indexed on Dno:
8 Total ≈ bE + bD = 505 block accesses

```

If Employee already carries a clustering index on Dno (plausible in practice, since Dno is a foreign key that's queried often), the Employee sort cost disappears and the total drops to roughly $bE + bD = 505$ block accesses - on par with block nested-loop.

(c) Hash Join

The partition phase hashes both relations into $nB-1 = 6$ buckets small enough to fit in memory, after which matching buckets are probed against each other.

```

1 Partition cost: 2×(bE + bD) = 2×(500 + 5) = 1,010
2 Probe cost: bE + bD = 500 + 5 = 505
3 Total = 1,515 block accesses

```

(d) Comparison and Recommendation

Join Method	Estimated Cost (block accesses)
Block Nested-Loop (Department as outer)	505
Hash Join	1,515
Sort-Merge Join (both unsorted)	3,515
Sort-Merge Join (Employee pre-sorted/indexed)	~505

Recommendation: given these statistics, block nested-loop join with the smaller relation (Department, 5 blocks) as the outer table is the cheapest option at roughly 505 block I/Os - essentially matched by sort-merge join only when Employee is already sorted or indexed on Dno. Hash join is a solid middle-ground choice when neither relation is sorted and memory is tight, since it sidesteps a full external sort. As a general rule: pick nested-loop when one relation is small enough to serve as the outer table within the buffer (or the inner relation's join attribute

is indexed); pick sort-merge when both inputs are already sorted, or will be reused sorted afterward; pick hash join for large, unsorted, unindexed equi-joins where memory can hold the smaller relation's partitions.

Q3. JDBC Connectivity Using PreparedStatement

The following program connects a Java application to a MySQL database (EmployeeDB) and runs a parameterized query that retrieves employees in a chosen department who earn above a chosen salary, relying on PreparedStatement to block SQL injection and to let the compiled query plan be reused safely.

```
1  import java.sql.*;
2
3  public class EmployeeQuery {
4
5      private static final String URL =
6          "jdbc:mysql://localhost:3306/EmployeeDB?useSSL=false&serverTimezone=UTC";
7      private static final String USER = "root";
8      private static final String PASSWORD = "your_password";
9
10     public static void main(String[] args) {
11
12         String sql = "SELECT Ename, Salary, Dno FROM Employee " +
13             "WHERE Dno = ? AND Salary > ?";
14
15         try (Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
16             PreparedStatement pstmt = con.prepareStatement(sql)) {
17
18             // Bind parameters positionally - never string-concatenate input
19             pstmt.setInt(1, 5);           // Dno
20             pstmt.setDouble(2, 50000.0); // Salary threshold
21
22             try (ResultSet rs = pstmt.executeQuery()) {
23                 while (rs.next()) {
24                     System.out.printf("%-15s %-10.2f %d\n",
25                         rs.getString("Ename"),
26                         rs.getDouble("Salary"),
27                         rs.getInt("Dno"));
28                 }
29             }
30
31         } catch (SQLException e) {
32             System.err.println("Database error: " + e.getMessage());
33             e.printStackTrace();
34         }
35     }
36 }
```

Running it from the terminal against EmployeeDB:

```
$ javac EmployeeQuery.java
> java EmployeeQuery
Ramesh Kumar      52000.00  5
Anitha Raj        61500.00  5
Suresh Babu       55250.00  5
```

Key points

- try-with-resources closes the Connection, PreparedStatement, and ResultSet automatically even when an exception occurs, so no database connections leak.
- The '?' placeholders are bound through typed setters (setInt, setDouble, setString, ...), letting the driver itself escape the values - this, not string concatenation, is what stops SQL injection.
- Because the SQL text stays fixed and only the parameter values change, MySQL can cache and reuse the compiled execution plan across calls, which beats re-parsing a fresh literal query every time.
- From MySQL Connector/J 6.0 onward the driver registers itself automatically through the JDBC 4.0 SPI, so an explicit Class.forName("com.mysql.cj.jdbc.Driver") call is optional, though it can still be kept for compatibility with older drivers.
- The same pattern works for INSERT/UPDATE/DELETE using pstmt.executeUpdate(); when the same statement runs across many rows, pstmt.addBatch()/executeBatch() should be used to cut down round-trips.

Q4. Conflict-Serializability via a Precedence Graph

Consider interleaved schedule S involving four transactions T1, T2, T3, and T4 operating on data items X, Y, and Z:

Step	T1	T2	T3	T4
1	Read(X)			
2		Read(Y)		
3	Write(X)			
4			Read(X)	
5		Write(Y)		
6			Write(X)	
7				Read(Y)
8				Write(Z)
9			Read(Z)	

Step 1 - Locate conflicting operation pairs

Two operations conflict when they come from different transactions, touch the same data item, and at least one of them is a write. Going through the schedule item by item:

- X: T1.Read(X) -> T1.Write(X) is within the same transaction, so it's ignored.
T1.Write(X) -> T3.Read(X) conflicts, with T1 preceding T3. T3.Read(X) -> T3.Write(X) is again within one transaction. T1.Write(X) -> T3.Write(X) also conflicts, T1 before T3.
- Y: T2.Read(Y) -> T2.Write(Y) is intra-transaction. T2.Write(Y) -> T4.Read(Y) conflicts, T2 before T4.
- Z: T4.Write(Z) -> T3.Read(Z) conflicts, T4 before T3.

Step 2 - Construct the precedence graph

One node is created per transaction, with a directed edge $T_i \rightarrow T_j$ drawn wherever an operation of T_i precedes and conflicts with an operation of T_j :

- $T_1 \rightarrow T_3$ (from both the Write(X)/Read(X) and Write(X)/Write(X) conflicts)
- $T_2 \rightarrow T_4$ (from the Write(Y)/Read(Y) conflict)
- $T_4 \rightarrow T_3$ (from the Write(Z)/Read(Z) conflict)

Step 3 - Check the graph for cycles

The edges are $\{T_1 \rightarrow T_3, T_2 \rightarrow T_4, T_4 \rightarrow T_3\}$. Following every path from each node - $T_1 \rightarrow T_3$ (a dead end) and $T_2 \rightarrow T_4 \rightarrow T_3$ (also a dead end) - no path ever loops back to a node it already visited, so the graph has no cycle.

Conclusion

Because the precedence graph is acyclic, schedule S qualifies as conflict-serializable. A topological ordering of the graph produces one valid equivalent serial order: T_2, T_1, T_4, T_3 (T_2 and T_1 have no incoming edges, so either can come first; T_4 must come after T_2 ; T_3 must come after both T_1 and T_4). Running the four transactions serially in this order would leave the database in exactly the same final state as the interleaved schedule S.

Q5. Basic Two-Phase Locking and Deadlock Identification

Consider two transactions running under Basic 2PL, where every lock request is granted right away unless it conflicts with a lock some other transaction already holds, and where a transaction may not acquire any new lock once it has released its first one (the core 2PL rule: all locking must precede all unlocking).

Time	T1	T2
t1	Lock-X(A)	
t2	Read(A)	
t3	Lock-X(B)	
t4	Read(B)	
t5	$A := A - 500$; Write(A)	
t6		$B := B + 500$; Write(B)
t7	Lock-X(B) -- BLOCKED (held by T2)	
t8		Lock-X(A) -- BLOCKED (held by T1)

Step 1 - Spot the deadlock

At t_7 , T_1 asks for Lock-X(B), but T_2 already holds it, so T_1 must wait. At t_8 , T_2 asks for Lock-X(A), but T_1 already holds it, so T_2 must wait too. T_1 is waiting on T_2 while T_2 is waiting on T_1 - a wait-for cycle $T_1 \rightarrow T_2 \rightarrow T_1$ - leaving both transactions permanently stuck. This is a textbook deadlock, and it's exactly the kind of situation Basic 2PL cannot prevent: 2PL guarantees conflict-serializability, not freedom from deadlock, because it never makes a transaction give up a lock it already holds - and here both transactions have already entered the locking phase while each still needs one more resource held by the other.

Step 2 - Resolve the deadlock

A deadlock detector in the lock manager (built periodically from the wait-for graph, or a timeout mechanism) finds the cycle and picks a victim to abort - usually whichever transaction has done the least work or carries the lowest priority. Suppose T2 is chosen:

1. Abort T2: undo its Write(B) (rolling B back to its value before t6) and release every lock T2 holds (Lock-X(B)).
2. T1's pending Lock-X(B) request can now be granted; T1 proceeds, writes B, and finally releases Lock-X(A) and Lock-X(B) together once it reaches its shrinking phase (commit).
3. T2 is restarted from scratch (often with an elevated priority so it isn't repeatedly picked as the victim and starved), redoing Lock-X(B), Read(B), Write(B), Commit.

After the rollback and restart, both transactions finish without further conflict, and the resulting execution (T1 in full, then the restarted T2 in full) is trivially serializable.

Q6. Wait-Die and Wound-Wait Deadlock Prevention

Both schemes rely on timestamps and are non-blocking by construction: each transaction gets a timestamp when it starts (an older transaction has a smaller timestamp and therefore higher priority), and that ordering decides, whenever a lock conflict arises, whether the requester waits or is aborted. Assume three transactions with start timestamps $TS(T1)=10$ (oldest), $TS(T2)=20$, and $TS(T3)=30$ (youngest), with T1 currently holding a lock on data item Q.

(a) Wait-Die scheme

Rule: if the requesting transaction T_i is older than the current holder T_j ($TS(T_i) < TS(T_j)$), T_i is permitted to wait. If T_i is younger ($TS(T_i) > TS(T_j)$), T_i 'dies' - it is aborted and later restarted with its original timestamp intact, so it eventually becomes the oldest transaction and is guaranteed to complete (no starvation).

Requester -> Holder	Timestamps	Comparison	Outcome
T2 requests Q, held by T1	$TS(T2)=20$, $TS(T1)=10$	T2 younger than T1	T2 dies (aborted, restarts later with $TS=20$)
T1 requests Q, held by T2 (later)	$TS(T1)=10$, $TS(T2)=20$	T1 older than T2	T1 waits
T3 requests Q, held by T1	$TS(T3)=30$, $TS(T1)=10$	T3 younger than T1	T3 dies (aborted, restarts with $TS=30$)

(b) Wound-Wait scheme

Rule: this mirrors Wait-Die. If the requester T_i is older than the holder T_j ($TS(T_i) < TS(T_j)$), T_i 'wounds' T_j - T_j is forcibly aborted and T_i takes over the lock (or briefly waits for that abort to finish). If T_i is younger than the holder, T_i simply waits.

Requester -> Holder	Timestamps	Comparison	Outcome
T2 requests Q, held by T1	$TS(T2)=20$, $TS(T1)=10$	T2 younger than T1	T2 waits
T1 requests Q, held by T2 (later)	$TS(T1)=10$, $TS(T2)=20$	T1 older than T2	T1 wounds T2 (T2 aborted, T1 proceeds)

T3 requests Q, held by T1	TS(T3)=30, TS(T1)=10	T3 younger than T1	T3 waits
---------------------------	----------------------	--------------------	----------

Why neither scheme allows deadlock

In both schemes, every wait relation follows one consistent global order (Wait-Die: an older transaction only ever waits for a younger one; Wound-Wait: a younger transaction only ever waits for an older one), so the wait-for graph can never form a cycle - a cycle would need some transaction to wait for another that is simultaneously waiting for it, which can't happen once waits are forced to flow in a single direction along the timestamp order. In practice, Wait-Die causes more rollbacks of the requesting transaction (the younger one dies), while Wound-Wait causes rollbacks of the holding transaction (the younger one is wounded) - Wound-Wait tends to need fewer restarts as a transaction ages, since older transactions increasingly push through rather than repeatedly dying.

Q7. Immediate Update Recovery with a Checkpoint

Under Immediate Update, a transaction's changes can be written to disk before it commits, so recovery needs both REDO (reapplying committed changes that never made it to disk) and UNDO (reversing uncommitted changes that did reach disk). Consider this log, which records a checkpoint while T1 was still active:

```
mysql> SELECT * FROM information_schema.INNODB_TRX_LOG;
<T1 start>
<T1, A, 1000, 900>
<T1, B, 2000, 2100>
<CHECKPOINT (active: T1)>
<T1 commit>
<T2 start>
<T2, C, 500, 400>
<T3 start>
<T3, D, 700, 800>
<T2 commit>
***** SYSTEM CRASH *****
```

Step 1 - Build the UNDO and REDO lists

1. Scan the log backward from the crash point to the checkpoint. Any transaction that shows a <Ti commit> record during that scan goes on the REDO list; any transaction still lacking a commit record once the checkpoint is reached goes on the UNDO list.
2. T1: started before the checkpoint and committed afterward -> REDO.
3. T2: started after the checkpoint and committed before the crash -> REDO.
4. T3: started after the checkpoint and never committed (no <T3 commit> appears before the crash) -> UNDO.

Step 2 - Carry out recovery

UNDO list = {T3}: T3's log records are processed in reverse order, restoring each item to its earlier value - D goes back from 800 to 700. (Under Immediate Update, T3's write may already sit on disk, so this step cannot be skipped.)

REDO list = {T1, T2}: their records are processed in forward (chronological) order, reapplying each new value in case the buffer manager hadn't flushed it before the crash - A becomes 900, B becomes 2100, C becomes 400.

Because the checkpoint shows T1 as still active, recovery cannot assume T1's pre-checkpoint writes were already safely on disk just because they're old - it redoes them regardless, which is safe since REDO is idempotent (reapplying a value that's already durable changes nothing).

Result after recovery

Item	Final Value	Reason
A	900	REDO (T1 committed)
B	2100	REDO (T1 committed)
C	400	REDO (T2 committed)
D	700	UNDO (T3 never committed)

Q8. Deferred Update (NO-UNDO/REDO) Recovery

Under Deferred Update, a transaction keeps all its writes in memory and only flushes them to disk once it commits - so an uncommitted transaction's changes are guaranteed to have never touched disk. This means recovery never has to UNDO anything; it only needs to REDO committed transactions whose writes might not yet be on disk. Consider this log:

```
mysql> SELECT * FROM information_schema.INNODB_TRX_LOG;
<T1 start>
<T1, X, 50>
<T1, Y, 80>
<T1 commit>
<T2 start>
<T2, X, 60>
<T3 start>
<T3, Z, 100>
<T2 commit>
***** SYSTEM CRASH ***** (T3 never committed)
```

Step 1 - Sort transactions

- T1: has a commit record -> REDO.
- T2: has a commit record -> REDO.
- T3: no commit record before the crash -> simply discarded. Because its buffered writes never reached disk, there is nothing to undo - its log entries are just skipped.

Step 2 - REDO pass

Scanning forward, the new value for every write belonging to a committed transaction is reapplied:

1. REDO <T1, X, 50>: set X = 50.
2. REDO <T1, Y, 80>: set Y = 80.
3. REDO <T2, X, 60>: set X = 60 (correctly overwriting T1's earlier value, reflecting execution order).
4. T3's write to Z is never applied - it's skipped entirely since T3 didn't commit.

Result after recovery

Item	Final Value	Reason
X	60	REDO; last committed writer was T2
Y	80	REDO (T1 committed)
Z	unchanged (pre-transaction value)	T3's write discarded - never committed

This is why the technique is called NO-UNDO/REDO: since uncommitted data can never reach the database, the recovery manager only ever performs REDO and never needs to reverse anything - making recovery simpler and faster than Immediate Update, at the cost of holding all of a transaction's updates in memory until it commits.

Q9. Strict 2PL Design for a Banking System (T1 Transfers, T2 Reads)

Scenario: T1 transfers Rs. 500 from Account A to Account B, while T2 concurrently reads the balances of A and B (say, to print a statement). Under plain 2PL, T2 could read A after T1 debits it but before T1 credits B, seeing an inconsistent halfway state if early lock release were allowed. Strict 2PL closes this gap with one extra rule layered on ordinary 2PL: a transaction keeps all of its exclusive (write) locks until it commits or aborts - locks are never released early, and certainly never before commit. This guarantees no other transaction can read or overwrite a value written by a still-active transaction, giving both conflict-serializability and recoverability (cascadeless schedules).

Design

1. T1 requests Lock-X(A). Granted, since no other transaction holds a lock on A.
2. T1 reads A, computes $A := A - 500$, and writes A. The exclusive lock on A stays in place under the strict rule - it is not released yet.
3. T1 requests Lock-X(B). Granted. T1 reads B, computes $B := B + 500$, and writes B.
4. Meanwhile T2 requests Lock-S(A) to read the balance. Since T1 still holds Lock-X(A) (strict 2PL hasn't released it), T2 is made to wait - it cannot glimpse A in its debited-but-not-yet-fully-transferred state.
5. T1 commits. Only at this point does the lock manager release Lock-X(A) and Lock-X(B) together.
6. T2's request for Lock-S(A) is now granted; T2 also obtains Lock-S(B), reads both post-transfer balances, and releases its shared locks once its own transaction finishes.

Why this meets the requirements

- Atomic visibility: T2 sees either both balances before the transfer or both after it, never a half-applied transfer, because A and B are unlocked together only at T1's commit.
- No cascading rollback: since T2 cannot read A or B before T1 commits, T2's result never depends on data from a transaction that might still abort - had T1 aborted instead, T2 simply would never have seen the uncommitted value, so no cascading undo of T2 is ever needed.
- Conflict-serializability: locking on every read/write, combined with the two-phase property (all acquisitions precede any release), guarantees the resulting schedule is equivalent to running T1 then T2 (or T2 then T1) serially.
- Deadlock note: with only these two transactions, and T2 only reading, no cycle can form here - but a general strict-2PL banking system should still run a wait-for-graph deadlock detector (or Wait-Die/Wound-Wait, as in Q6), since concurrent transfers touching the same pair of accounts in opposite orders could deadlock.

Q10. Pseudocode for Heuristic Query Optimization

The routine below implements the three classic heuristic rules named in the question - push selections down, project as early as possible, and order joins so the cheapest (smallest expected intermediate result) join runs first - as a transformation applied to a canonical relational-algebra query tree.

```

1  FUNCTION OptimizeQueryTree(tree):
2
3      // ---- Rule 1: Push-down selections ----
4      FOR EACH selection node sigma(condition) in tree:
5          IF condition is a conjunction (c1 AND c2 AND ... AND cn):
6              SPLIT into a cascade of single-condition selections
7              sigma_c1( sigma_c2( ... sigma_cn( child ) ... ) )
8          FOR EACH single-condition selection sigma_c:
9              WHILE sigma_c's child is NOT a leaf
10                 AND all attributes in c belong to ONE side
11                 of the child operator (join / product):
12                     MOVE sigma_c below that operator, next to the
13                     relevant child (leaf relation or subtree)
14          IF a selection's condition references attributes from
15          BOTH sides of a Cartesian product below it:
16              REPLACE sigma_condition(A x B) WITH A join_condition B
17
18      // ---- Rule 2: Push-down / early projection ----
19      FOR EACH leaf relation R in tree:
20          neededAttrs = attributes of R referenced anywhere
21                       above this leaf (later selections, joins,
22                       groupings, or the final SELECT list)
23          IF neededAttrs is a strict subset of R's schema:
24              INSERT pi_neededAttrs(R) directly above the leaf
25      FOR EACH intermediate node in tree (bottom-up):
26          IF columns flowing out exceed what any ancestor needs:
27              INSERT a projection immediately above this node
28
29      // ---- Rule 3: Order joins by smallest intermediate result ----
30      joinNodes = COLLECT all join / product nodes in tree
31      FOR EACH joinNode in joinNodes:
32          estimate = EstimateSize(joinNode.left) *
33                   EstimateSize(joinNode.right) *
34                   SelectivityOf(joinNode.condition)
35          joinNode.estimatedCost = estimate
36      SORT joinNodes ascending by estimatedCost
37      REBUILD the join sequence left-deep, cheapest join first
38
39      RETURN tree // optimized, heuristically-ordered query tree
40
41
42  FUNCTION EstimateSize(node):
43      IF node is a leaf relation R: RETURN |R|
44      IF node is sigma_condition(child):
45          RETURN EstimateSize(child) * SelectivityOf(condition)
46      IF node is a join/product of (left, right):
47          RETURN EstimateSize(left) * EstimateSize(right)
48          * SelectivityOf(joinCondition)

```

Design notes

- The three rules run in this deliberate order: selections and projections are pushed down first because that shrinks every relation before join ordering estimates their sizes - ordering joins based on un-filtered relation sizes would give the wrong answer.
- SelectivityOf(condition) relies on standard estimates (for instance, $1/|\text{distinct values}|$ for an equality predicate, or a fixed fraction such as $1/3$ for a range predicate when no histogram exists), letting EstimateSize be computed without actually running the query.
- Building the plan left-deep rather than bushy keeps the search space manageable and lets each join immediately reuse the previous join's pipelined output - which is why most heuristic optimizers default to left-deep plans before a cost-based optimizer goes on to explore bushier alternatives.